

Introduction to Machine Learning for Text Analysis with Python

Day 5 – Friday

» Navigating the Model Ecosystem, Critical Evaluation, optional topics + Project Day 2«

Rupert Kiddle
Sjoerd Stolwijk

r.t.kiddle@vu.nl, @rptkiddle
s.b.stolwijk@uu.nl

Today

Recap

Model selection

class imbalance

Pitfalls

Clean, high-quality code

Project day 2

Your takeaway

Recap

Model selection

Evaluation criteria for computational approaches

Method-immanent criteria

- **Accuracy** against a gold standard
- **Speed** of processing
- **Transparency** of the model's inner workings
- **Replicability** of classification results
- **Language coverage** — esp. non-English texts

Method-external criteria

- **Resources & competences** needed to apply the method
- **Ethical questions**, incl. external costs to nature & society:
 - energy consumption per task
 - data privacy / ownership rights

Source: Maurer et al., 2026

Showstoppers - Example

	Dictionary		SML	UML	DL	LLM	gLLM	
	off-the-shelf	custom-made					local	API
Data & task characteristics								
no (large) annotated dataset available	✓	✓	✗	✓	✗	✓	✓	✓
no language-specific tool available	✗	✓	✓	✓	✓	✗	✗	✗
categories known a-priori	✓	✓	✓	✗	✓	✓	✓	✓
categories not known a-priori	✗	✗	✗	✓	✗	✓	✓	✓
Resources & skills required								
no access to high-performance computing	✓	✓	✓	✓	(✓)	(✗)	(✗)	✓
no programming skills	(✓)	(✓)	✗	✗	✗	✗	✗	(✓)
no statistics skills	✓	(✗)	✗	✗	✗	✗	✓	✓

Showstoppers - Example

	Dictionary		SML	UML	DL	LLM	gLLM	
	off-the-shelf	custom-made					local	API
Show stopper:								
Data & task characteristics								
no (large) annotated dataset available	✓	✓	✗	✓	✗	✓	✓	✓
no language-specific tool available	✗	✓	✓	✓	✓	✗	✗	✗
categories known a-priori	✓	✓	✓	✗	✓	✓	✓	✓
categories not known a-priori	✗	✗	✗	✓	✗	✓	✓	✓
Resources & skills required								
no access to high-performance computing	✓	✓	✓	✓	(✓)	(✗)	(✗)	✓
no programming skills	(✓)	(✓)	✗	✗	✗	✗	✗	(✓)
no statistics skills	✓	(✗)	✗	✗	✗	✗	✓	✓

Showstoppers - Example

	Dictionary		SML	UML	DL	LLM	gLLM	
	off-the-shelf	custom-made					local	API
Show stopper:								
Data & task characteristics								
no (large) annotated dataset available	✓	✓	✗	✓	✗	✓	✓	✓
no language-specific tool available	✗	✓	✓	✓	✓	✗	✗	✗
categories known a-priori	✓	✓	✓	✗	✓	✓	✓	✓
categories not known a-priori	✗	✗	✗	✓	✗	✓	✓	✓
Resources & skills required								
no access to high-performance computing	✓	✓	✓	✓	(✓)	(✗)	(✗)	✓
no programming skills	(✓)	(✓)	✗	✗	✗	✗	✗	(✓)
no statistics skills	✓	(✗)	✗	✗	✗	✗	✓	✓

Performance: How good is good enough

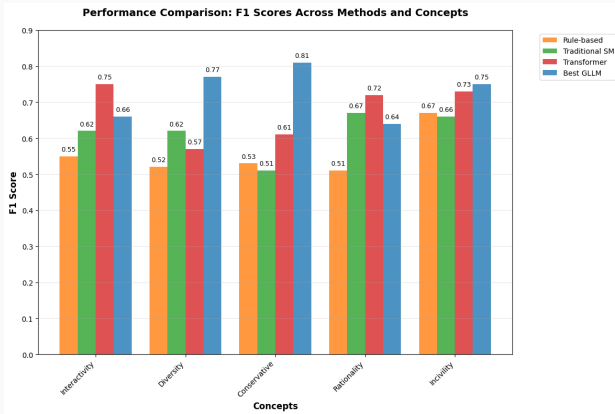


Figure 1: Overview of differences in macro average F1-macro scores for different classification approaches reported in Stolwijk et al., 2026

not all text needs automated annotation

Remember sampling theory from statistics: more is not always better!

⇒ Do you really need to code so much data, or does a sample suffice

class imbalance

Sampling

Sampling

First, a stratified random sample was drawn from the large databases of tweets and *YouTube* posts of the TV shows. For this selection of *Twitter* and *YouTube* posts by the TV shows, we then retrieved the earliest comments (up to 20 maximum). The sample was stratified according to the three news genres that were included, so this was evenly distributed among regular news, partisan news, and news satire shows.

Figure 2: Appendix in Stolwijk et al., 2026

Training class imbalance

To address the issue of strong class imbalance, we have implemented a WeightedLossTrainer class during the training phase to account for disparities in class representation. Additionally, and for most concepts, optimize training based on the F1 score of the minority class during training. For incivility and liberal, this strategy proved insufficient. Here, we used a down-sampling strategy for the majority classes in the training dataset and subsequently optimized training on a weighted F1 score. We performed hyperparameter tuning, including exploring a range of learning rates and batch sizes. We

Figure 3: Appendix in Stolwijk et al., 2026

Pitfalls

Things to watch out for

- Label leakage
- Multiple comparisons: use your test set only once
- Old cached results - running notebook out of order
- Old cached results - not cleaning your model after a training run
- Typos and errors
- Quality of your manual baseline
- Good performance $\neq$ same result
- Free AI / compute often means data sharing

The dangers of AI - yesterday's slide

	Pred.: non-offensive	Pred.: offensive
non-offensive	506	114
offensive	87	153

Class 1 (Offensive) Metrics:

$$\text{Recall} = \frac{153}{153 + 87} = \frac{153}{240} \approx 0.638$$

$$\text{Precision} = \frac{153}{153 + 114} = \frac{153}{267} \approx 0.573$$

$$\text{F1} = 2 \cdot \frac{0.573 \times 0.638}{0.573 + 0.638} \approx 0.604$$

Claude Haiku's Confusion Matrix

Predicted	0: non-offensive	1: offensive
0: non-offensive	506	114
1: offensive	87	153

Class 1 (Offensive) Metrics:

$$\text{Recall} = \frac{153}{153 + 114} = \frac{153}{267} \approx 0.573$$

$$\text{Precision} = \frac{153}{153 + 87} = \frac{153}{240} \approx 0.638$$

$$\text{F1} = 2 \cdot \frac{0.638 \times 0.573}{0.638 + 0.573} \approx 0.604$$

Making Mistakes: You Are Not Alone

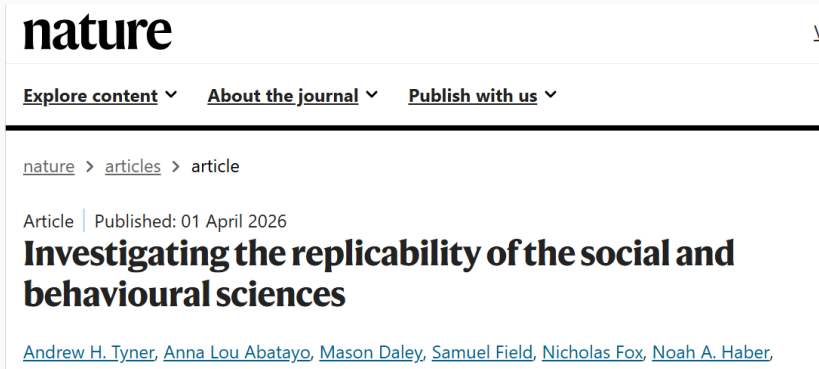


Figure 4: Replications showed statistically significant results in the original pattern for 151 of 274 claims (55.1% [95% CI 49.2–60.9%]) and for 80.8 of 164 papers (49.3% [95% CI 43.8–54.7%]), weighed for replicating multiple claims per paper (Tyner et al., 2026)

Bias: Good performance $\neq$ same answer - Prevalence

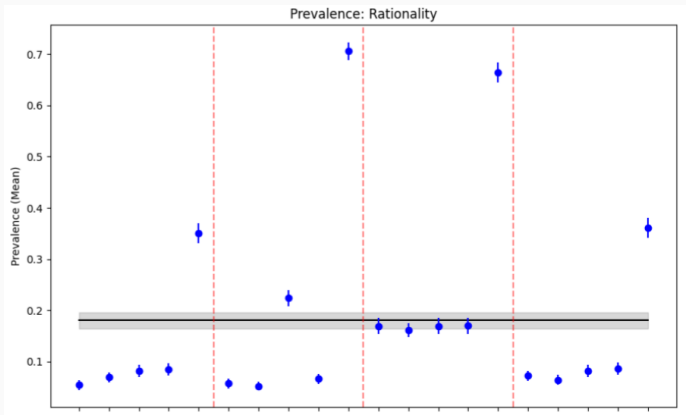


Figure 5: Overview of rationality prevalence in corpus according to different GLLMs relative to manual data reported in Stolwijk et al., 2025

Bias: Good performance $\neq$ same answer - Correlations

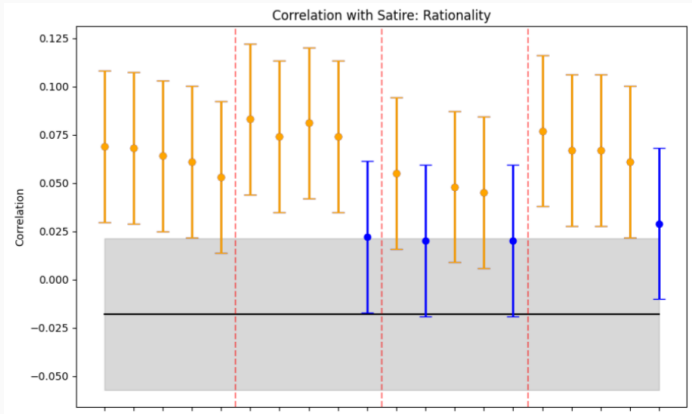


Figure 6: Overview of correlations genre-rationality according to different GLLMs relative to manual data reported in Stolwijk et al., 2025

Good performance $\neq$ same answer - across models and concepts

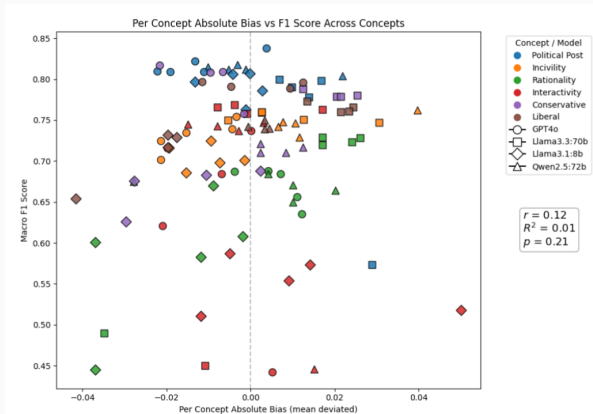


Figure 7: F1 versus correlation bias across concepts and GLLMs, relative to manual data reported in Stolwijk et al., 2025

Good performance $\neq$ same answer - biased annotations

- simulation errors versus F1 across -

Good performance $\neq$ same answer - biased GLLMs

- graph inter- and intra-prompt agreement –

Dealing with bias

Three emerging approaches to deal with bias

- Design-Based Supervised Learning (DSL; Egami et al., 2024)
- Prediction-Powered Inference (PPI; Angelopoulos et al., 2023)
- Maximum Likelihood Adjustment (MLA; TeBlunthuis et al., 2024)

Remember the gold standard: TweetEval-offensive dataset

We annotated the dataset using the crowdsourcing platform Figure Eight.

We ensured annotation quality by:

- Hiring only experienced annotators
- Using test questions to filter annotators
- Requiring two annotations per tweet
- Requesting a third annotation in case of disagreement
- Using majority vote for final labels

Source: Zampieri et al., 2019

What to do about all this some suggestions

- Being aware of these dangers is half the work
- Always validate a classifier for your task
- Use high-quality manual annotations
- Start your notebook by separating the random test set and keep it isolated until the end.
- Use GitHub or equivalent; if AI messes up you can go back
- If possible, share data and models
- Annotate and document your code
- Let AI check your code for errors

What to do about all this some suggestions

- Being aware of these dangers is half the work
- Always validate a classifier for your task
- Use high-quality manual annotations
- Start your notebook by separating the random test set and keep it isolated until the end.
- Use GitHub or equivalent; if AI messes up you can go back
- If possible, share data and models
- Annotate and document your code
- Let AI check your code for errors

What to do about all this some suggestions

- Being aware of these dangers is half the work
- Always validate a classifier for your task
- Use high-quality manual annotations
- Start your notebook by separating the random test set and keep it isolated until the end.
- Use GitHub or equivalent; if AI messes up you can go back
- If possible, share data and models
- Annotate and document your code
- Let AI check your code for errors

What to do about all this some suggestions

- Being aware of these dangers is half the work
- Always validate a classifier for your task
- Use high-quality manual annotations
- Start your notebook by separating the random test set and keep it isolated until the end.
- Use GitHub or equivalent; if AI messes up you can go back
- If possible, share data and models
- Annotate and document your code
- Let AI check your code for errors

What to do about all this some suggestions

- Being aware of these dangers is half the work
- Always validate a classifier for your task
- Use high-quality manual annotations
- Start your notebook by separating the random test set and keep it isolated until the end.
- Use GitHub or equivalent; if AI messes up you can go back
 - If possible, share data and models
 - Annotate and document your code
 - Let AI check your code for errors

What to do about all this some suggestions

- Being aware of these dangers is half the work
- Always validate a classifier for your task
- Use high-quality manual annotations
- Start your notebook by separating the random test set and keep it isolated until the end.
- Use GitHub or equivalent; if AI messes up you can go back
- If possible, share data and models
- Annotate and document your code
- Let AI check your code for errors

What to do about all this some suggestions

- Being aware of these dangers is half the work
- Always validate a classifier for your task
- Use high-quality manual annotations
- Start your notebook by separating the random test set and keep it isolated until the end.
- Use GitHub or equivalent; if AI messes up you can go back
- If possible, share data and models
- Annotate and document your code
- Let AI check your code for errors

What to do about all this some suggestions

- Being aware of these dangers is half the work
- Always validate a classifier for your task
- Use high-quality manual annotations
- Start your notebook by separating the random test set and keep it isolated until the end.
- Use GitHub or equivalent; if AI messes up you can go back
- If possible, share data and models
- Annotate and document your code
- Let AI check your code for errors

Pitfalls

Clean, high-quality code

Develop components separately

One script for downloading the data, one script for analyzing

- Avoids waste of resources (e.g., unnecessary downloading multiple times)
- Makes it easier to re-use your code or apply it to other data

Start small, then scale up

- Start with a small and simple dataset, problem, or algorithm to get a sense of what you're doing, and what the ML is of all involved
- Then, gradually increase the size of the dataset, the complexity of the problem, and the size of the algorithm

- This helps you to understand the problem and the data better

- This helps you to understand the problem and the data better

Develop components separately

One script for downloading the data, one script for analyzing

- Avoids waste of resources (e.g., unnecessary downloading multiple times)
- Makes it easier to re-use your code or apply it to other data

Start small, then scale up

- Start with a small dataset and a simple model to get a baseline performance
- Then, gradually increase the complexity of the model and the size of the dataset
- Monitor the performance and adjust the model and dataset size accordingly

Develop components separately

One script for downloading the data, one script for analyzing

- Avoids waste of resources (e.g., unnecessary downloading multiple times)
- Makes it easier to re-use your code or apply it to other data

Start small, then scale up

- Start with a small dataset and a simple model to get a baseline performance
- Then, gradually increase the complexity of the model and the size of the dataset
- Monitor the performance and adjust the model and dataset size accordingly

Develop components separately

One script for downloading the data, one script for analyzing

- Avoids waste of resources (e.g., unnecessary downloading multiple times)
- Makes it easier to re-use your code or apply it to other data

Start small, then scale up

- Take your plan and solve *one* problem at a time (e.g., parsing a review page; or getting the URLs of all review pages)
- Test each step in your code with a small test sample
- Print() all intermediate results for sanity checks

Develop components separately

One script for downloading the data, one script for analyzing

- Avoids waste of resources (e.g., unnecessary downloading multiple times)
- Makes it easier to re-use your code or apply it to other data

Start small, then scale up

- Take your plan and solve *one* problem at a time (e.g., parsing a review page; or getting the URLs of all review pages)
- Test each step in your code with a small test sample
- `Print()` all intermediate results for sanity checks

Develop components separately

One script for downloading the data, one script for analyzing

- Avoids waste of resources (e.g., unnecessary downloading multiple times)
- Makes it easier to re-use your code or apply it to other data

Start small, then scale up

- Take your plan and solve *one* problem at a time (e.g., parsing a review page; or getting the URLs of all review pages)
- Test each step in your code with a small test sample
- Print() all intermediate results for sanity checks

Develop components separately

If you copy-paste code your own code, you are doing something wrong

- Use loops!
- If something takes more than a couple of lines, write a function!

Develop components separately

If you copy-paste code your own code, you are doing something wrong

- Use loops!
- If something takes more than a couple of lines, write a function!

Make it robust

You cannot foresee every possible problem.

Most important: Make sure your program does not fail and you lose all data just because something goes wrong at case 997/1000.

- Use an environment
- Track package versions or use UV
- Document, document, document -> you won't remember what you did many commits down the line
- Use try/except to explicitly tell the program how to handle errors
- Write data to files (or database) in between

Project day 2

Your takeaway

Recap
○

Model selection
○○○○○

class imbalance
○○○

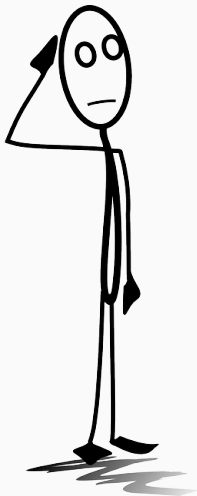
Pitfalls
○○○○○○○○○○○○○○○○○○○○

Project day 2
○

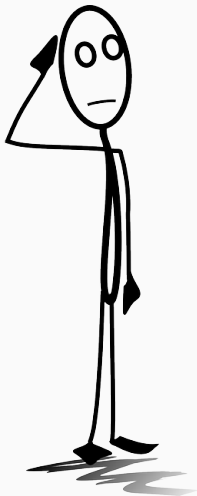
Your takeaway
●○○○

References
○

(short recap of course)



Have your plans about how to and whether to use ML changed?



What are your next steps?

Recap
○

Model selection
○○○○○

class imbalance
○○○

Pitfalls
○○○○○○○○○○○○○○○○○○○○

Project day 2
○

Your takeaway
○○○○●

References
○

Last part: we help you work on (or discuss)
your own projects.

References



Angelopoulos, A. N., Bates, S., Fannjiang, C., Jordan, M. I., & Zrnic, T. (2023). **Prediction-powered inference**. *Science*, 382(6671), 669–674. <https://doi.org/10.1126/science.adi6000>



Egami, N., Hinck, M., Stewart, B. M., & Wei, H. (2024). **Using large language model annotations for the social sciences: A general framework of using predicted variables in downstream analyses**. *Preprint from November, 17, 2024*. Retrieved August 21, 2025, from https://naokiegami.com/paper/dsl_ss.pdf



Maurer, S., Trilling, et al. (2026). ***Dictionaries, machine learning, transformers or generative LLMs? A framework for deciding which approach to use for computational opinion mining*** [Working paper, COST Action OPINION (CA21129), WG Tools].



Stolwijk, S. B., Boukes, M., & Trilling, D. (2025). **Are generative ai text annotations systematically biased?** <https://arxiv.org/abs/2512.08404>



Stolwijk, S. B., Boukes, M., Yeung, W. N., Liao, Y., Münker, S., Kroon, A. C., & Trilling, D. (2026). **Can we use automated approaches to measure the quality of online political discussion? how to (not) measure interactivity, diversity, rationality, and incivility in online comments to the news.** *Communication Methods and Measures*, 20(1), 1–25.
<https://doi.org/10.1080/19312458.2025.2553300>



TeBlunthuis, N., Hase, V., & Chan, C.-H. (2024). **Misclassification in Automated Content Analysis Causes Bias in Regression. Can We Fix It? Yes We Can!** *Communication Methods and Measures*, 18(3), 278–299.
<https://doi.org/10.1080/19312458.2023.2293713>



Tyner, A. H., Abatayo, A. L., Daley, M., Field, S., Fox, N., Haber, N. A., Hahn, K. M., Struhl, M. K., Mawhinney, B., Miske, O., et al. (2026). **Investigating the replicability of the social and behavioural sciences.** *Nature*, 652(8108), 143–150.



Zampieri, M., Malmasi, S., Nakov, P., Rosenthal, S., Farra, N., & Kumar, R. (2019, June). **SemEval-2019 task 6: Identifying and categorizing offensive language in social media (OffensEval)**. In J. May, E. Shutova, A. Herbelot, X. Zhu, M. Apidianaki, & S. M. Mohammad (Eds.), *Proceedings of the 13th international workshop on semantic evaluation* (pp. 75–86). Association for Computational Linguistics. <https://doi.org/10.18653/v1/S19-2010>